FIAP · TESTES AUTOMATIZADOS

Projeto — Cidades ESGInteligentes

Testes BDD de API da Eco Compliance API: Gherkin, Reqnroll e contratos JSON Schema

Integrante	**Luiz Alberto Silva de Santana** (RM565199)
Aplicação	API REST em C# / ASP.NET Core 8 de compliance ambiental (ESG)
Ferramentas	Reqnroll 3.3 (BDD, sucessor do SpecFlow), Gherkin em português, xUnit, JsonSchema.Net
Repositório	github.com/Daikan1/eco-compliance-dotnet (branch `feature/bdd-tests`, PR #1)
Execução	Local via Docker (`./scripts/run-bdd-tests.sh`) e no GitHub Actions (job "Testes BDD (API)")

33
cenários executados

15 / 18
positivos / negativos

13 / 13
endpoints testados

100%
aprovados (local e CI)

Sumário

1. Objetivo e relação com os pilares ESG
2. Estratégia e arquitetura dos testes
3. Cenários de teste em Gherkin (detalhados por funcionalidade)
4. Testes de API: status code, corpo JSON e contrato (JSON Schema)
5. Configuração: prints
6. Como executar
7. Evidências de execução
8. Adaptações na aplicação e desafios
9. Atendimento aos requisitos

1. Objetivo e relação com os pilares ESG

A Eco Compliance API registra empresas, indicadores ambientais (emissões, energia, resíduos) e obrigações de compliance. Erros nesses dados comprometem relatórios de sustentabilidade e auditorias. Os testes automatizados garantem que o sistema se comporta corretamente nos caminhos felizes e, principalmente, que **recusa operações inválidas**.

Pilar / tema	Como os testes contribuem
Governança e compliance	Garantem integridade e rastreabilidade dos registros de compliance: criação vinculada a uma empresa existente, atualização de status, remoção controlada e acesso restrito a usuários autenticados (JWT).
Ambiental	Validam o registro de indicadores de emissões (tCO2e), energia (kWh) e resíduos reciclados (kg). Um indicador com **valor negativo** é recusado, porque não pode distorcer o inventário ambiental.
Eficiência	A suíte roda em cerca de 2 s com banco em memória e sobe só o necessário, o que economiza recursos de CI a cada commit.
Rastreabilidade	Os contratos exigem datas em `date-time` RFC 3339 (com fuso), e `GET /` informa ambiente e versão em execução.

2. Estratégia e arquitetura dos testes

Gherkin (.feature)
Cenários em português, escritos na linguagem do negócio

→

Reqnroll
Liga cada passo a um *step definition* em C#

→

HttpClient
Requisições HTTP reais para a API

→

API em memória
`WebApplicationFactory` + EF Core InMemory (1 banco por cenário)

→

Validações
Status code, campos do JSON e contrato JSON Schema

Decisão	Motivo
Reqnroll em C#	É o sucessor oficial do SpecFlow, que foi descontinuado. Mantém testes e aplicação na mesma linguagem, solução e pipeline.
Gherkin em português (`# language: pt`)	Cenários legíveis por pessoas de negócio e auditoria, não só por desenvolvedores.
API real em memória	Passa por todo o pipeline HTTP de produção (rotas, JWT, validação, middleware de erros e serialização) sem depender do Oracle da FIAP, que se mostrou instável. Os testes ficam determinísticos.
Banco isolado por cenário	Nenhum cenário interfere em outro, e os testes podem rodar em qualquer ordem.
JSON Schema draft 2020-12	Contratos com `additionalProperties: false` : um campo novo, removido ou com tipo diferente quebra o teste.

Estrutura do projeto de testes

```
EcoCompliance.BddTests/
├── Features/      5 arquivos .feature (Gherkin em português)
├── Steps/         ApiSteps, EmpresaSteps, ComplianceSteps, IndicadorSteps, RespostaSteps
├── Schemas/       11 contratos JSON Schema (*.schema.json)
├── Support/       EcoApiFactory (API em memória), ApiContext (estado do cenário), ContractValidator
└── reqnroll.json  idioma pt-BR e relatório HTML (TestResults/relatorio-bdd.html)
```

3. Cenários de teste em Gherkin

Os **33 cenários** estão distribuídos em 5 funcionalidades. Cada uma usa o formato *Como / Quero / Para* e tem cenários positivos (caminho feliz) e negativos (falhas esperadas). Os *Esquemas do Cenário* usam tabelas de *Exemplos* para repetir o mesmo comportamento com dados diferentes.

Funcionalidade	Pilar ESG	Cenários
`MonitoramentoApi.feature`	Governança: rastreabilidade de deploys e disponibilidade	**2**
`Autenticacao.feature`	Governança: controle de acesso a dados ESG	**4**
`Empresas.feature`	Governança: cadastro das organizações monitoradas	**8**
`ComplianceAmbiental.feature`	Governança e compliance ambiental: integridade e rastreabilidade	**12**
`IndicadoresEsg.feature`	Ambiental: emissões, energia e resíduos	**7**
Total		**33**

Nas páginas seguintes, cada funcionalidade traz uma tabela-resumo (tipo, status esperado e contrato validado) e o arquivo `.feature` completo.

3.1 Monitoramento da API

Arquivo: `Features/MonitoramentoApi.feature` · **Pilar ESG:** Governança: rastreabilidade de deploys e disponibilidade · **2 cenários**

Cenário	Tipo	Status	Contrato validado
Consultar a identificação da API em execução	**Positivo**	**200**	`app-info`
Verificar a saúde da aplicação	**Positivo**	**200**	—

```gherkin
# language: pt
@monitoramento
Funcionalidade: Monitoramento da API
  Como equipe de operações responsável pela governança do sistema ESG
  Quero saber qual versão está no ar e se a aplicação está saudável
  Para garantir rastreabilidade dos deploys e disponibilidade do serviço

  Cenário: Consultar a identificação da API em execução
    Quando consulto a identificação da API
    Então o status da resposta deve ser 200
    E a resposta deve seguir o contrato "app-info"
    E o campo "environment" da resposta deve ser "Testing"
    E o campo "application" da resposta deve conter "Cidades ESG Inteligentes"

  Cenário: Verificar a saúde da aplicação
    Quando consulto o health check
    Então o status da resposta deve ser 200
    E o corpo da resposta deve ser "Healthy"
```

3.2 Autenticação de gestores ESG

Arquivo: `Features/Autenticacao.feature` · **Pilar ESG:** Governança: controle de acesso a dados ESG · **4 cenários**

Cenário	Tipo	Status	Contrato validado
Login com credenciais válidas retorna um token JWT	**Positivo**	**200**	`login`
Login com senha incorreta é recusado	**Negativo**	**401**	`auth-error`
Login sem informar a senha é rejeitado pela validação	**Negativo**	**400**	`validation-problem`
Operação protegida sem token é bloqueada	**Negativo**	**401**	—

```
# language: pt
@autenticacao
Funcionalidade: Autenticação de gestores ESG
  Como gestor de compliance ambiental
  Quero me autenticar na API com usuário e senha
  Para que apenas pessoas autorizadas alterem dados de sustentabilidade (pilar de Governança)

  Cenário: Login com credenciais válidas retorna um token JWT
    Quando faço login com as credenciais do administrador
    Então o status da resposta deve ser 200
    E a resposta deve seguir o contrato "login"
    E o campo "token" da resposta deve estar preenchido

  Cenário: Login com senha incorreta é recusado
    Quando faço login com usuário "admin" e senha "senha-errada"
    Então o status da resposta deve ser 401
    E a resposta deve seguir o contrato "auth-error"
    E o campo "message" da resposta deve ser "Invalid credentials"

  Cenário: Login sem informar a senha é rejeitado pela validação
    Quando envio uma requisição POST para "/api/auth/login" com o corpo:
      """
      { "username": "admin" }
      """
    Então o status da resposta deve ser 400
    E a resposta deve seguir o contrato "validation-problem"
    E o erro de validação deve apontar o campo "Password"

  Cenário: Operação protegida sem token é bloqueada
    Dado que não estou autenticado
    Quando cadastro a empresa "Empresa Sem Login"
    Então o status da resposta deve ser 401
    E a resposta deve exigir autenticação do tipo "Bearer"
```

3.3 Cadastro de empresas monitoradas

Arquivo: `Features/Empresas.feature` · **Pilar ESG:** Governança: cadastro das organizações monitoradas · **8 cenários**

Cenário	Tipo	Status	Contrato validado
Cadastrar uma empresa com sucesso	**Positivo**	**201**	`company`
Cadastrar uma empresa sem nome é rejeitado	**Negativo**	**400**	`validation-problem`
Consultar uma empresa cadastrada pelo id	**Positivo**	**200**	`company`
Consultar uma empresa inexistente retorna 404	**Negativo**	**404**	`error`
Listar empresas com paginação	**Positivo**	**200**	`company-page`
Paginação inválida é recusada 3 exemplos	**Negativo**	**400**	`error`

```
# language: pt
@empresas
Funcionalidade: Cadastro de empresas monitoradas
  Como gestor de compliance ambiental
  Quero cadastrar e consultar as empresas acompanhadas pela plataforma
  Para vincular a elas os indicadores ESG e os registros de conformidade

  Contexto:
    Dado que estou autenticado como administrador

  Cenário: Cadastrar uma empresa com sucesso
    Quando cadastro a empresa "Prefeitura de São Paulo" com CNPJ "46395000000139" do setor "Governo"
    Então o status da resposta deve ser 201
    E a resposta deve seguir o contrato "company"
    E o campo "name" da resposta deve ser "Prefeitura de São Paulo"
    E o campo "sector" da resposta deve ser "Governo"
    E a resposta deve indicar a localização do recurso criado

  Cenário: Cadastrar uma empresa sem nome é rejeitado
    Quando envio uma requisição POST para "/api/companies" com o corpo:
      """
      { "cnpj": "46395000000139", "sector": "Governo" }
      """
    Então o status da resposta deve ser 400
    E a resposta deve seguir o contrato "validation-problem"
    E o erro de validação deve apontar o campo "Name"

  Cenário: Consultar uma empresa cadastrada pelo id
    Dado que existe a empresa "Secretaria do Verde e Meio Ambiente" cadastrada
    Quando consulto a empresa "Secretaria do Verde e Meio Ambiente"
    Então o status da resposta deve ser 200
    E a resposta deve seguir o contrato "company"
    E o campo "name" da resposta deve ser "Secretaria do Verde e Meio Ambiente"

  Cenário: Consultar uma empresa inexistente retorna 404
    Quando consulto a empresa de id 999
    Então o status da resposta deve ser 404
    E a resposta deve seguir o contrato "error"
    E o campo "message" da resposta deve ser "Company not found with id: 999"

  Cenário: Listar empresas com paginação
    Dado que existem as empresas:
      | nome                    |
      | Cooperativa Recicla Mais |
      | Usina Solar Horizonte    |
      | Transportes Verdes S.A.  |
    Quando listo as empresas com page 1 e pageSize 2
    Então o status da resposta deve ser 200
    E a resposta deve seguir o contrato "company-page"
    E a resposta deve conter 2 itens
    E o campo "totalCount" da resposta deve ser 3
    E o campo "totalPages" da resposta deve ser 2

  Esquema do Cenário: Paginação inválida é recusada
    Quando listo as empresas com page <page> e pageSize <pageSize>
    Então o status da resposta deve ser 400
```

```
    E a resposta deve seguir o contrato "error"
    E o campo "message" da resposta deve conter "<campo>"

    Exemplos:
      | page | pageSize | campo    |
      | 0    | 10       | page     |
      | 1    | 0        | pageSize |
      | 1    | 101      | pageSize |
```

3.4 Rastreabilidade do compliance ambiental

Arquivo: `Features/ComplianceAmbiental.feature` · **Pilar ESG:** Governança e compliance ambiental: integridade e rastreabilidade · **12 cenários**

Cenário	Tipo	Status	Contrato validado
Registrar uma obrigação de compliance para uma empresa	Positivo	201	`compliance`
Registrar compliance para uma empresa inexistente	Negativo	404	`error`
Registrar compliance sem o tipo da obrigação é rejeitado	Negativo	400	`validation-problem`
Atualizar o status de um registro mantém a rastreabilidade	Positivo	200	`compliance`
Atualizar um registro de compliance inexistente	Negativo	404	`error`
Remover um registro de compliance	Positivo	204	—
Remover um registro de compliance inexistente	Negativo	404	`error`
Listar todos os registros de compliance	Positivo	200	`compliance-page`
Listar os registros de compliance de uma empresa	Positivo	200	`compliance-page`
Listar compliance de uma empresa inexistente	Negativo	404	`error`
Alterações de compliance exigem autenticação 2 exemplos	Negativo	401	—

```
# language: pt
@compliance
Funcionalidade: Rastreabilidade do compliance ambiental
  Como auditor de conformidade regulatória
  Quero registrar, atualizar e remover registros de compliance de cada empresa
  Para manter um histórico íntegro e rastreável das obrigações ambientais

  Contexto:
    Dado que estou autenticado como administrador
    E que existe a empresa "Prefeitura de São Paulo" cadastrada

  Cenário: Registrar uma obrigação de compliance para uma empresa
    Quando registro o compliance "Licença Ambiental de Operação" com status "EM_ANALISE" para a empresa "Prefeitura
de São Paulo"
    Então o status da resposta deve ser 201
    E a resposta deve seguir o contrato "compliance"
    E o campo "companyName" da resposta deve ser "Prefeitura de São Paulo"
    E o campo "status" da resposta deve ser "EM_ANALISE"

  Cenário: Registrar compliance para uma empresa inexistente
    Quando registro o compliance "Inventário de Emissões GEE" com status "PENDENTE" para a empresa de id 999
    Então o status da resposta deve ser 404
    E a resposta deve seguir o contrato "error"
    E o campo "message" da resposta deve ser "Company not found with id: 999"

  Cenário: Registrar compliance sem o tipo da obrigação é rejeitado
    Quando envio uma requisição POST para "/api/compliance" com o corpo:
      """
      { "companyId": 1, "status": "PENDENTE", "reportDate": "2026-10-01T00:00:00Z" }
      """
    Então o status da resposta deve ser 400
    E a resposta deve seguir o contrato "validation-problem"
    E o erro de validação deve apontar o campo "ComplianceType"

  Cenário: Atualizar o status de um registro mantém a rastreabilidade
    Dado que existe o compliance "Plano de Gerenciamento de Resíduos" com status "EM_ANALISE" para a empresa
"Prefeitura de São Paulo"
    Quando atualizo o status desse registro de compliance para "CONFORME"
    Então o status da resposta deve ser 200
    E a resposta deve seguir o contrato "compliance"
    E o campo "status" da resposta deve ser "CONFORME"
    E o campo "complianceType" da resposta deve ser "Plano de Gerenciamento de Resíduos"

  Cenário: Atualizar um registro de compliance inexistente
```

```
    Quando atualizo o registro de compliance de id 999 para a empresa "Prefeitura de São Paulo"
    Então o status da resposta deve ser 404
    E a resposta deve seguir o contrato "error"
    E o campo "message" da resposta deve ser "ComplianceRecord not found with id: 999"

  Cenário: Remover um registro de compliance
    Dado que existe o compliance "Outorga de Uso da Água" com status "VENCIDO" para a empresa "Prefeitura de São
Paulo"
    Quando removo esse registro de compliance
    Então o status da resposta deve ser 204
    E ao listar os registros de compliance da empresa "Prefeitura de São Paulo" a resposta deve conter 0 itens

  Cenário: Remover um registro de compliance inexistente
    Quando removo o registro de compliance de id 999
    Então o status da resposta deve ser 404
    E a resposta deve seguir o contrato "error"

  Cenário: Listar todos os registros de compliance
    Dado que existe o compliance "Licença Ambiental de Operação" com status "CONFORME" para a empresa "Prefeitura
de São Paulo"
    E que existe o compliance "Inventário de Emissões GEE" com status "EM_ANALISE" para a empresa "Prefeitura de
São Paulo"
    Quando listo os registros de compliance
    Então o status da resposta deve ser 200
    E a resposta deve seguir o contrato "compliance-page"
    E a resposta deve conter 2 itens

  Cenário: Listar os registros de compliance de uma empresa
    Dado que existe a empresa "Usina Solar Horizonte" cadastrada
    E que existe o compliance "Licença Ambiental de Operação" com status "CONFORME" para a empresa "Prefeitura de
São Paulo"
    E que existe o compliance "Relatório de Sustentabilidade" com status "EM_ANALISE" para a empresa "Usina Solar
Horizonte"
    Quando listo os registros de compliance da empresa "Usina Solar Horizonte"
    Então o status da resposta deve ser 200
    E a resposta deve seguir o contrato "compliance-page"
    E a resposta deve conter 1 item
    E o campo "items[0].companyName" da resposta deve ser "Usina Solar Horizonte"

  Cenário: Listar compliance de uma empresa inexistente
    Quando listo os registros de compliance da empresa de id 999
    Então o status da resposta deve ser 404
    E a resposta deve seguir o contrato "error"

  Esquema do Cenário: Alterações de compliance exigem autenticação
    Dado que existe o compliance "Licença Ambiental de Operação" com status "CONFORME" para a empresa "Prefeitura
de São Paulo"
    E que não estou autenticado
    Quando envio uma requisição <método> para o registro de compliance criado
    Então o status da resposta deve ser 401

    Exemplos:
      | método |
      | PUT    |
      | DELETE |
```

3.5 Registro de indicadores ESG

Arquivo: `Features/IndicadoresEsg.feature` · **Pilar ESG:** Ambiental: emissões, energia e resíduos · **7 cenários**

Cenário	Tipo	Status	Contrato validado
Registrar indicadores ambientais (3 exemplos)	Positivo	201	`esg-indicator`
Indicador com valor negativo é rejeitado (integridade dos dados ESG)	Negativo	400	`validation-problem`
Indicador para uma empresa inexistente	Negativo	404	`error`
Listar os indicadores ESG registrados	Positivo	200	`esg-indicator-page`
Registrar indicador sem autenticação é bloqueado	Negativo	401	—

```
# language: pt
@indicadores
Funcionalidade: Registro de indicadores ESG
  Como analista de sustentabilidade
  Quero registrar indicadores ambientais medidos em cada empresa
  Para acompanhar emissões, consumo de energia e reciclagem de resíduos

  Contexto:
    Dado que estou autenticado como administrador
    E que existe a empresa "Cooperativa Recicla Mais" cadastrada

  Esquema do Cenário: Registrar indicadores ambientais
    Quando registro o indicador "<indicador>" com valor <valor> "<unidade>" para a empresa "Cooperativa Recicla
Mais"
    Então o status da resposta deve ser 201
    E a resposta deve seguir o contrato "esg-indicator"
    E o campo "indicatorType" da resposta deve ser "<indicador>"
    E o campo "value" da resposta deve ser <valor>
    E o campo "companyName" da resposta deve ser "Cooperativa Recicla Mais"

    Exemplos:
      | indicador              | valor  | unidade |
      | EMISSOES_CO2           | 120.5  | tCO2e   |
      | CONSUMO_ENERGIA        | 3400   | kWh     |
      | RESIDUOS_RECICLADOS    | 850.75 | kg      |

  Cenário: Indicador com valor negativo é rejeitado (integridade dos dados ESG)
    Quando registro o indicador "EMISSOES_CO2" com valor -50 "tCO2e" para a empresa "Cooperativa Recicla Mais"
    Então o status da resposta deve ser 400
    E a resposta deve seguir o contrato "validation-problem"
    E o erro de validação deve apontar o campo "Value"

  Cenário: Indicador para uma empresa inexistente
    Quando registro o indicador "CONSUMO_AGUA" com valor 1500 "m3" para a empresa de id 999
    Então o status da resposta deve ser 404
    E a resposta deve seguir o contrato "error"
    E o campo "message" da resposta deve ser "Company not found with id: 999"

  Cenário: Listar os indicadores ESG registrados
    Dado que existe o indicador "EMISSOES_CO2" com valor 98.2 "tCO2e" para a empresa "Cooperativa Recicla Mais"
    E que existe o indicador "RESIDUOS_RECICLADOS" com valor 430 "kg" para a empresa "Cooperativa Recicla Mais"
    Quando listo os indicadores ESG
    Então o status da resposta deve ser 200
    E a resposta deve seguir o contrato "esg-indicator-page"
    E a resposta deve conter 2 itens

  Cenário: Registrar indicador sem autenticação é bloqueado
    Dado que não estou autenticado
    Quando registro o indicador "EMISSOES_CO2" com valor 10 "tCO2e" para a empresa "Cooperativa Recicla Mais"
    Então o status da resposta deve ser 401
```

4. Testes de API: status code, corpo JSON e contrato

Todos os **13 endpoints** da API são testados. Cada cenário valida o **status code**, campos do **corpo JSON** (ex.: `o campo "status" da resposta deve ser "CONFORME"`) e, quando há corpo JSON, o **contrato** por JSON Schema.

Endpoint	Status validados	Contratos
`GET /`	200	app-info
`GET /health`	200 (corpo `Healthy`)	texto puro
`POST /api/auth/login`	200 · 400 · 401	login · validation-problem · auth-error
`POST /api/companies`	201 · 400 · 401	company · validation-problem
`GET /api/companies/{id}`	200 · 404	company · error
`GET /api/companies`	200 · 400	company-page · error
`POST /api/compliance`	201 · 400 · 404	compliance · validation-problem · error
`GET /api/compliance`	200	compliance-page
`GET /api/compliance/company/{id}`	200 · 404	compliance-page · error
`PUT /api/compliance/{id}`	200 · 401 · 404	compliance · error
`DELETE /api/compliance/{id}`	204 · 401 · 404	error
`GET /api/esg-indicators`	200	esg-indicator-page
`POST /api/esg-indicators`	201 · 400 · 401 · 404	esg-indicator · validation-problem · error

Contratos (JSON Schema)

Contrato	Usado em	Regras principais
`app-info`	GET /	application, environment, version e links fixos /swagger e /health
`login`	POST /api/auth/login (200)	token no formato JWT (3 partes base64url) e expiresAt em date-time
`auth-error`	POST /api/auth/login (401)	message obrigatória
`company / company-page`	Empresas	id ≥ 1, name não vazio, cnpj e sector string ou null; página com items, totalCount, page, pageSize (1–100) e totalPages
`compliance / compliance-page`	Compliance	companyId, companyName, complianceType, status e reportDate (date-time RFC 3339)
`esg-indicator / esg-indicator-page`	Indicadores ESG	value numérico ≥ 0, unit string ou null, measuredAt (date-time)
`error`	Erros 400/404 do middleware	timestamp (date-time), status ∈ {400, 404, 500}, error e message
`validation-problem`	Erros de validação (RFC 7807)	status = 400 e errors com ao menos um campo e mensagem

Todos os contratos usam draft 2020-12 e validação de formato ativada (`RequireFormatValidation`). O validador mostra o caminho exato do campo inválido (ex.: `/name: Value is "string" but should be "integer"`).

5. Configuração

Configuração do projeto de testes — Reqnroll + xUnit + JSON Schema

EcoCompliance.BddTests/EcoCompliance.BddTests.csproj

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <RootNamespace>EcoCompliance.BddTests</RootNamespace>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\EcoCompliance.API\EcoCompliance.API.csproj" />
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="JsonSchema.Net" Version="9.4.1" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.10" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="8.0.10" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.11.0" />
    <PackageReference Include="Reqnroll.xUnit" Version="3.3.4" />
    <PackageReference Include="xunit" Version="2.9.3" />
    <PackageReference Include="xunit.runner.visualstudio" Version="4.0.1">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
  </ItemGroup>

  <ItemGroup>
    <None Update="reqnroll.json" CopyToOutputDirectory="PreserveNewest" />
    <None Include="Schemas\*.schema.json" CopyToOutputDirectory="PreserveNewest" />
  </ItemGroup>

</Project>
```

EcoCompliance.BddTests/reqnroll.json

```json
{
  "$schema": "https://schemas.reqnroll.net/reqnroll-config-latest.json",
  "language": {
    "feature": "pt-BR",
    "binding": "en-US"
  },
  "formatters": {
    "html": {
      "outputFilePath": "../../../../TestResults/relatorio-bdd.html"
    },
```

Figura 1: Projeto de testes. Pacotes Reqnroll.xUnit, JsonSchema.Net, Mvc.Testing e EF Core InMemory; `reqnroll.json` com Gherkin em pt-BR, cultura en-US para números e relatório HTML.

Execução — Dockerfile.tests e job de CI (GitHub Actions)

Dockerfile.tests

```dockerfile
# ─────────────────────────────────────────────────────────────────────────────
# Imagem para executar os testes BDD (Reqnroll + Gherkin) de API em .NET 8,
# sem precisar instalar nada além do Docker.
#
#   docker build -f Dockerfile.tests -t eco-compliance-bdd .
#   docker run --rm -v "$PWD/TestResults:/src/TestResults" eco-compliance-bdd
#
# Relatórios gerados em ./TestResults: relatorio-bdd.html e bdd-results.trx
# ─────────────────────────────────────────────────────────────────────────────
FROM mcr.microsoft.com/dotnet/sdk:8.0
WORKDIR /src

# Restore separado do código-fonte: aproveita o cache de camadas
COPY ["EcoCompliance.API/EcoCompliance.API.csproj", "EcoCompliance.API/"]
COPY ["EcoCompliance.BddTests/EcoCompliance.BddTests.csproj", "EcoCompliance.BddTests/"]
RUN dotnet restore "EcoCompliance.BddTests/EcoCompliance.BddTests.csproj"

COPY EcoCompliance.API/ EcoCompliance.API/
COPY EcoCompliance.BddTests/ EcoCompliance.BddTests/
RUN dotnet build "EcoCompliance.BddTests/EcoCompliance.BddTests.csproj" -c Release --no-restore

ENTRYPOINT ["dotnet", "test", "EcoCompliance.BddTests/EcoCompliance.BddTests.csproj", \
            "-c", "Release", "--no-build", \
            "--logger", "trx;LogFileName=bdd-results.trx", \
            "--logger", "console;verbosity=normal", \
            "--results-directory", "/src/TestResults"]
```

.github/workflows/ci-cd.yml (linhas 59–95)

```yaml
  bdd-tests:
    name: Testes BDD (API)
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: ${{ env.DOTNET_VERSION }}

      - name: Restore
        run: dotnet restore EcoCompliance.BddTests/EcoCompliance.BddTests.csproj

      - name: Build
        run: dotnet build EcoCompliance.BddTests/EcoCompliance.BddTests.csproj -c Release --no-restore

      - name: Cenários BDD (Gherkin)
        run: >
          dotnet test EcoCompliance.BddTests/EcoCompliance.BddTests.csproj -c Release --no-build
          --logger "trx;LogFileName=bdd-results.trx"
          --logger "console;verbosity=normal"
          --results-directory TestResults

      - name: Resumo dos cenários
        if: always()
        run: python3 scripts/bdd-summary.py TestResults/bdd-results.trx >> "$GITHUB_STEP_SUMMARY"

      - name: Publicar relatório BDD (HTML + TRX)
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: relatorio-bdd
          path: TestResults
```

Figura 2: `Dockerfile.tests` (execução com um comando, só com Docker) e o job **Testes BDD (API)** no GitHub Actions, que publica o resumo e o relatório HTML como artefato.

API em memória por cenário e contrato JSON Schema

EcoCompliance.BddTests/Support/EcoApiFactory.cs (linhas 16–40)

```
public sealed class EcoApiFactory : WebApplicationFactory<Program>
{
    public const string AdminUser     = "admin";
    public const string AdminPassword = "Senha-Testes-BDD-2026";
    private const string JwtKey       = "chave-jwt-exclusiva-dos-testes-bdd-com-mais-de-32-caracteres";

    private readonly string _databaseName = $"eco-bdd-{Guid.NewGuid()}";

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Testing");
        builder.UseSetting("Auth:Username", AdminUser);
        builder.UseSetting("Auth:Password", AdminPassword);
        builder.UseSetting("Auth:JwtKey", JwtKey);
        builder.UseSetting("Database:ApplyMigrations", "false");

class=c>        // Erros esperados (400/404) não poluem a saída dos testes
        builder.ConfigureLogging(logging => logging.ClearProviders());

        builder.ConfigureServices(services =>
        {
            services.RemoveAll<DbContextOptions<AppDbContext>>();
            services.AddDbContext<AppDbContext>(options => options.UseInMemoryDatabase(_databaseName));
        });
    }
```

EcoCompliance.BddTests/Schemas/compliance.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Registro de compliance",
  "type": "object",
  "properties": {
    "id": {
      "type": "integer",
      "minimum": 1
    },
    "companyId": {
      "type": "integer",
      "minimum": 1
    },
    "companyName": {
      "type": "string"
    },
    "complianceType": {
      "type": "string",
      "minLength": 1
    },
    "status": {
      "type": "string",
      "minLength": 1
    },
    "reportDate": {
      "type": "string",
      "format": "date-time"
    }
  },
  "required": [
    "id",
    "companyId",
    "companyName",
    "complianceType",
    "status",
    "reportDate"
  ],
  "additionalProperties": false
}
```

Figura 3: `EcoApiFactory` sobe a API real com credenciais de teste e banco InMemory isolado; abaixo, o contrato `compliance.schema.json`.

6. Como executar

Forma	Comando	Requisito
Docker (recomendado)	`./scripts/run-bdd-tests.sh`	Docker
.NET local	`./scripts/run-bdd-tests.sh --local`	.NET 8 SDK
Docker manual	`docker build -f Dockerfile.tests -t eco-compliance-bdd .` `docker run --rm -v "$PWD/TestResults:/src/TestResults" eco-compliance-bdd`	Docker
CI	Automático em todo push e pull request para a `main`	—

Saídas em `TestResults/`: `relatorio-bdd.html` (relatório navegável com cada passo), `bdd-results.trx` e o resumo em Markdown impresso no terminal (e na página da execução no CI).

7. Evidências de execução

Cenários Gherkin — Features/ComplianceAmbiental.feature

EcoCompliance.BddTests/Features/ComplianceAmbiental.feature (linhas 1–45)

```gherkin
# language: pt
@compliance
Funcionalidade: Rastreabilidade do compliance ambiental
  Como auditor de conformidade regulatória
  Quero registrar, atualizar e remover registros de compliance de cada empresa
  Para manter um histórico íntegro e rastreável das obrigações ambientais

  Contexto:
    Dado que estou autenticado como administrador
    E que existe a empresa "Prefeitura de São Paulo" cadastrada

  Cenário: Registrar uma obrigação de compliance para uma empresa
    Quando registro o compliance "Licença Ambiental de Operação" com status "EM_ANALISE" para a empresa "Prefeitura de São Paulo"
    Então o status da resposta deve ser 201
    E a resposta deve seguir o contrato "compliance"
    E o campo "companyName" da resposta deve ser "Prefeitura de São Paulo"
    E o campo "status" da resposta deve ser "EM_ANALISE"

  Cenário: Registrar compliance para uma empresa inexistente
    Quando registro o compliance "Inventário de Emissões GEE" com status "PENDENTE" para a empresa de id 999
    Então o status da resposta deve ser 404
    E a resposta deve seguir o contrato "error"
    E o campo "message" da resposta deve ser "Company not found with id: 999"

  Cenário: Registrar compliance sem o tipo da obrigação é rejeitado
    Quando envio uma requisição POST para "/api/compliance" com o corpo:
      """
      { "companyId": 1, "status": "PENDENTE", "reportDate": "2026-10-01T00:00:00Z" }
      """
    Então o status da resposta deve ser 400
    E a resposta deve seguir o contrato "validation-problem"
    E o erro de validação deve apontar o campo "ComplianceType"

  Cenário: Atualizar o status de um registro mantém a rastreabilidade
    Dado que existe o compliance "Plano de Gerenciamento de Resíduos" com status "EM_ANALISE" para a empresa "Prefeitura de São Paulo"
    Quando atualizo o status desse registro de compliance para "CONFORME"
    Então o status da resposta deve ser 200
    E a resposta deve seguir o contrato "compliance"
    E o campo "status" da resposta deve ser "CONFORME"
    E o campo "complianceType" da resposta deve ser "Plano de Gerenciamento de Resíduos"

  Cenário: Atualizar um registro de compliance inexistente
    Quando atualizo o registro de compliance de id 999 para a empresa "Prefeitura de São Paulo"
    Então o status da resposta deve ser 404
    E a resposta deve seguir o contrato "error"
```

Figura 4: Trecho de `ComplianceAmbiental.feature`: cenários positivos e negativos de rastreabilidade do compliance.

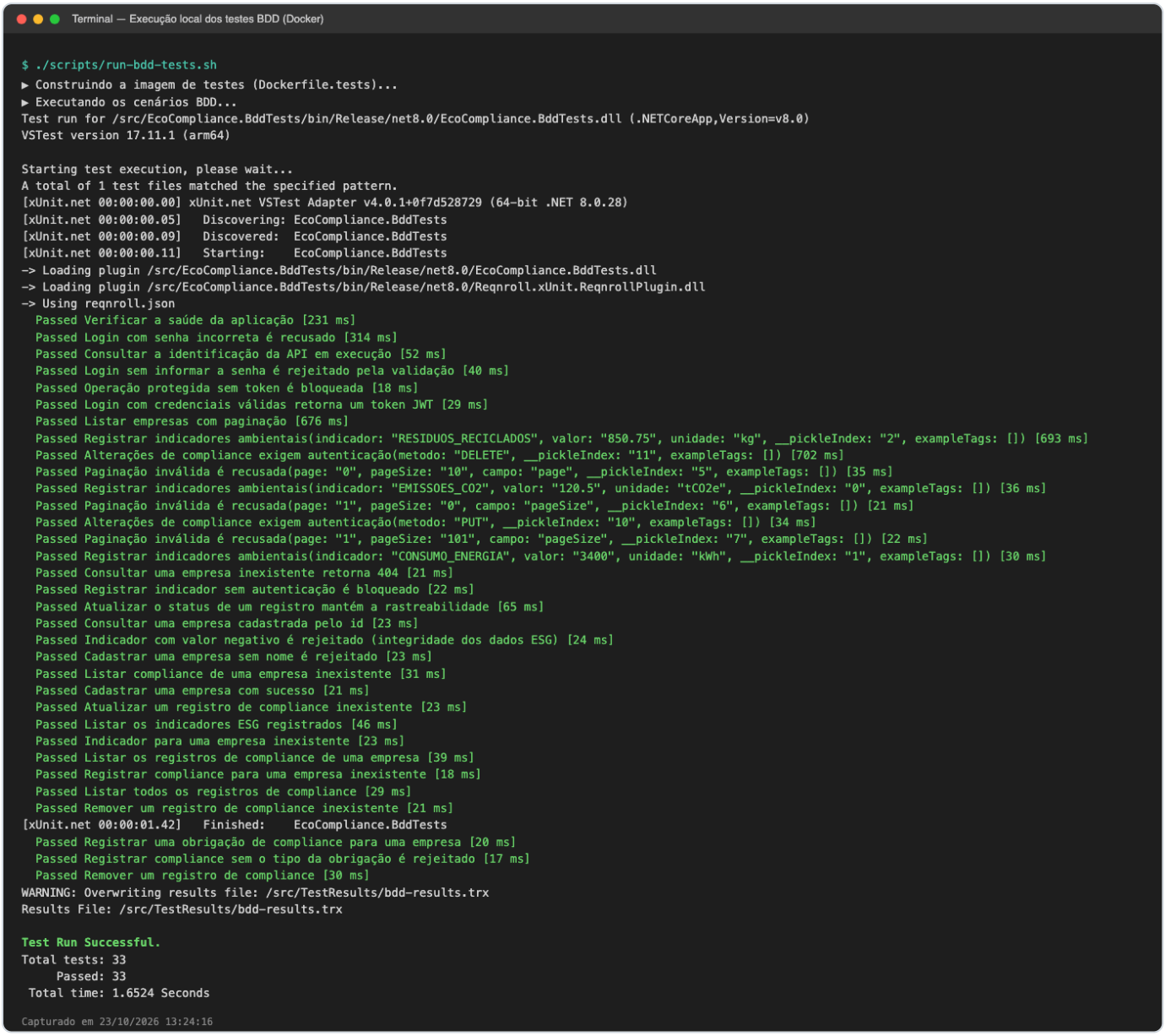


Figura 5: Execução local com `./scripts/run-bdd-tests.sh` em Docker (.NET 8): **Test Run Successful, 33 aprovados**.

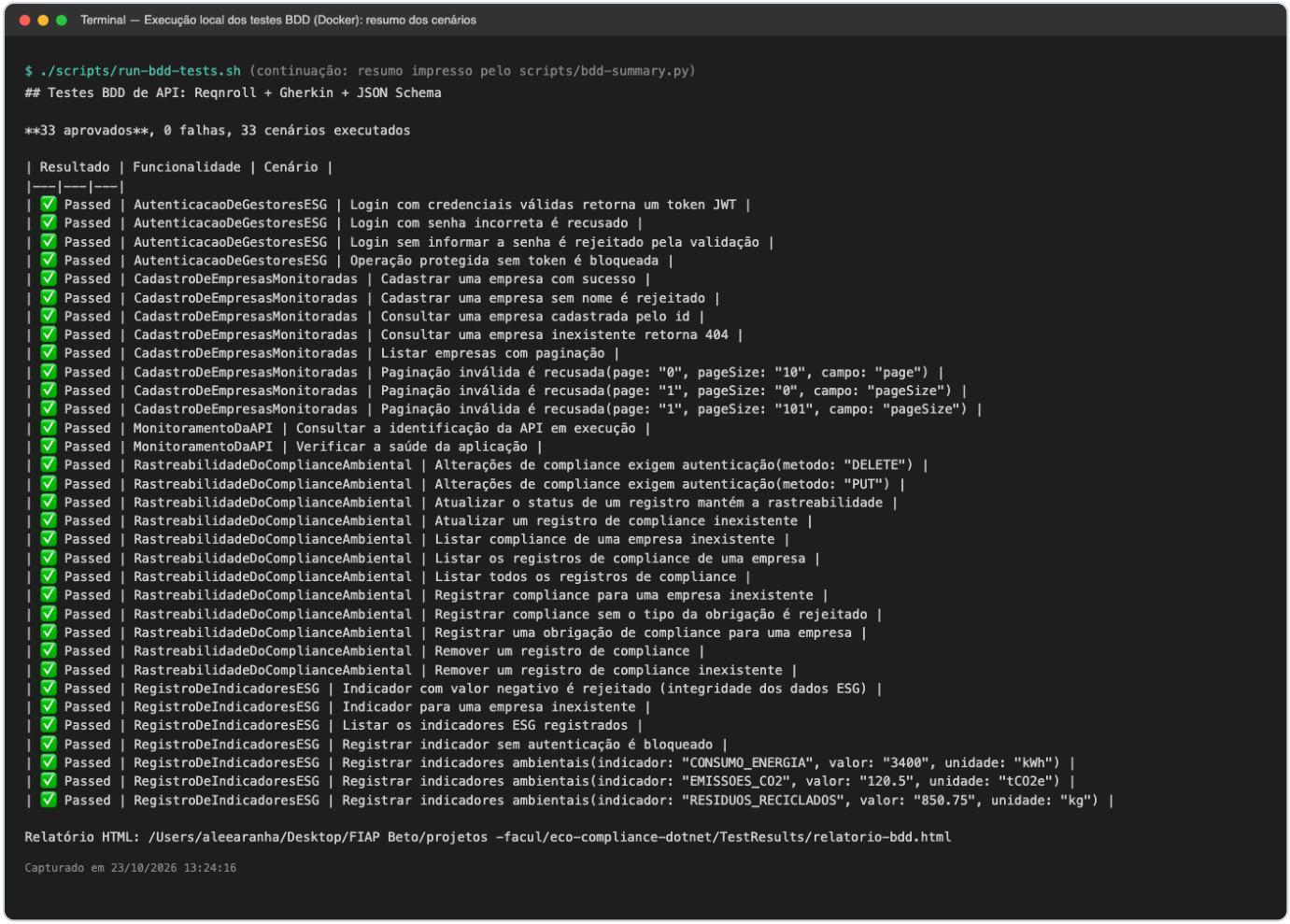


Figura 6: Resumo impresso pelo script ao final da mesma execução: resultado de cada um dos 33 cenários por funcionalidade.

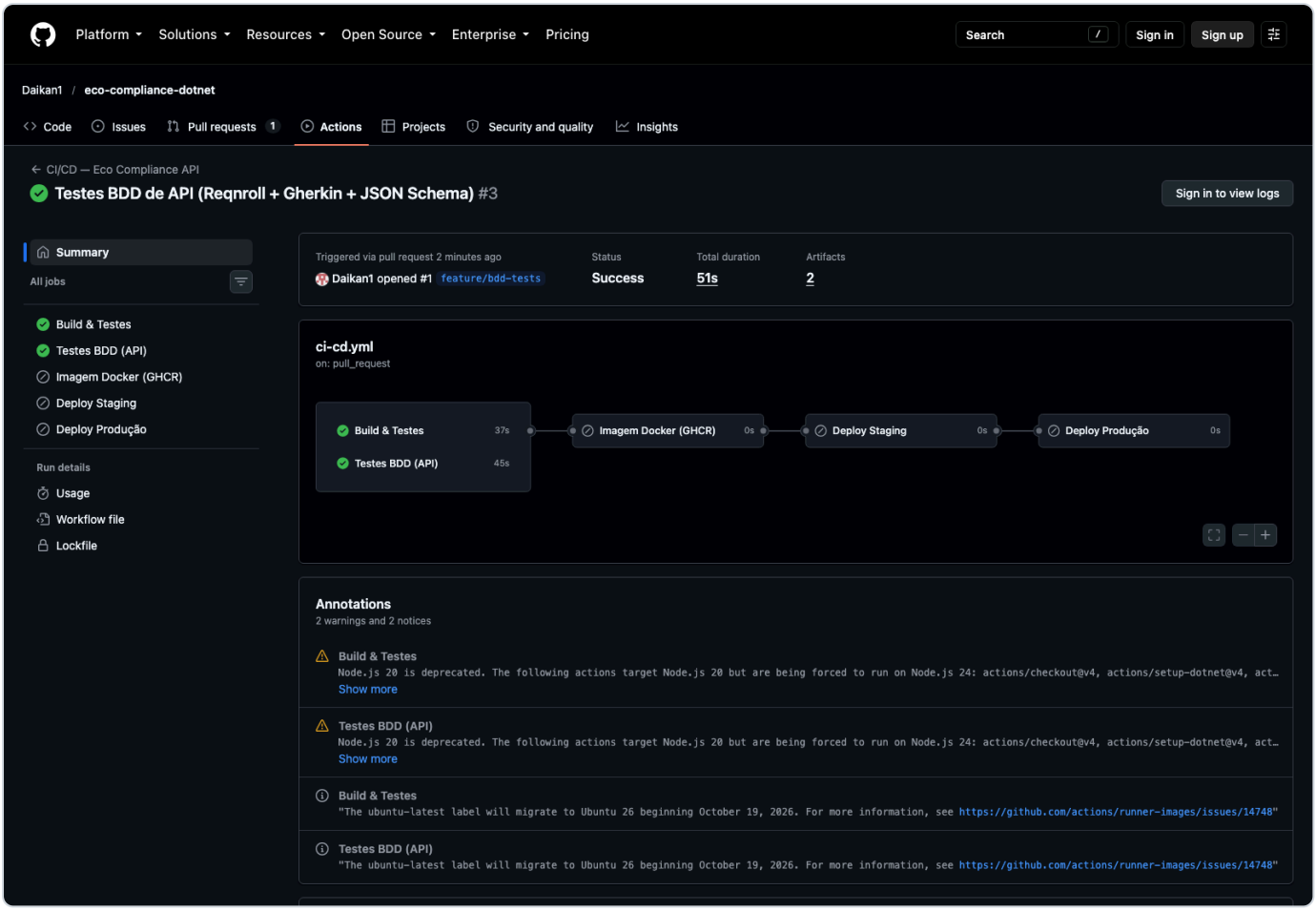


Figura 7: Pipeline no GitHub Actions (PR #1): **Build & Testes** e **Testes BDD (API)** aprovados; os jobs de deploy são pulados em pull request, por design. Artefato `relatorio-bdd` publicado.

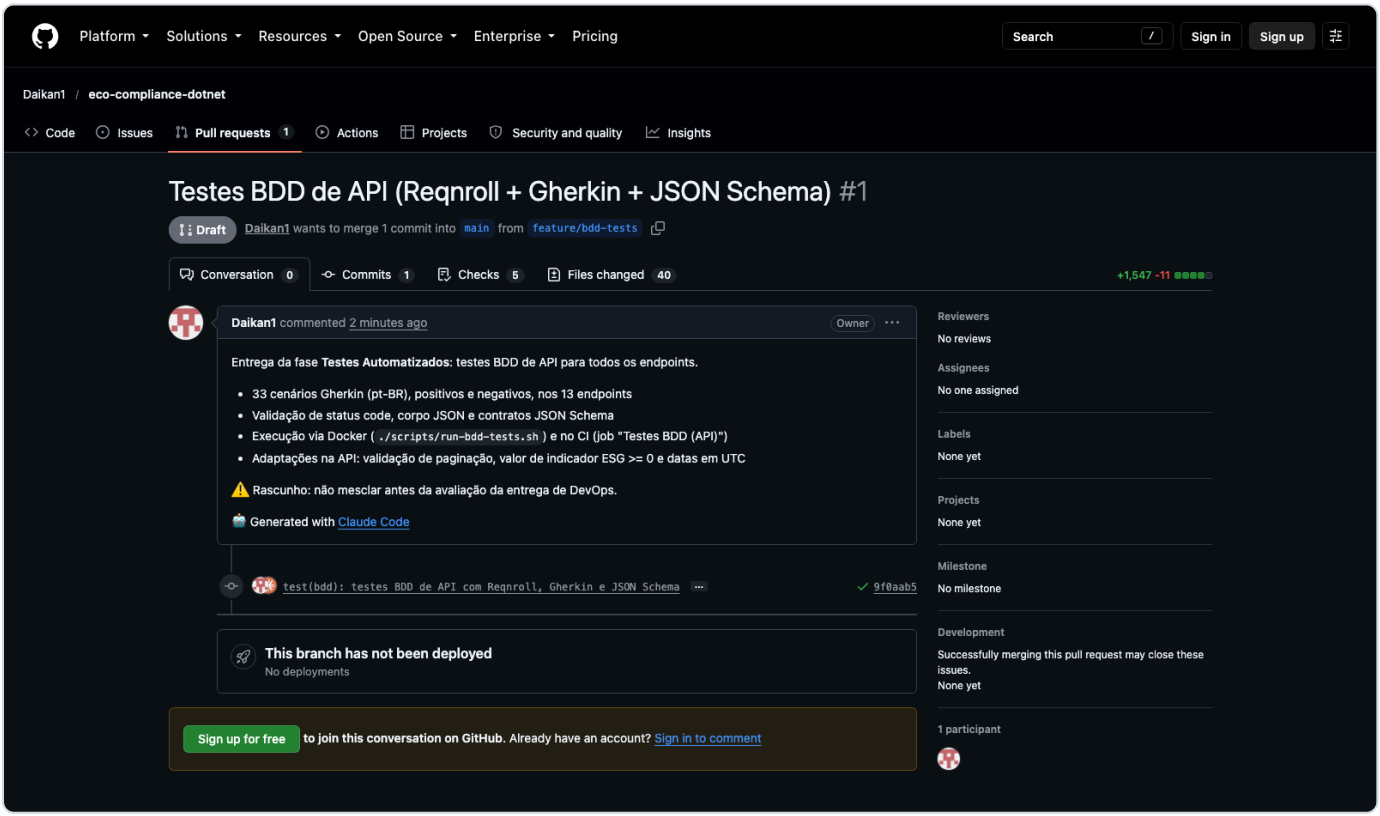


Figura 8: Pull request #1 (rascunho) com os 5 checks do pipeline. A branch não foi implantada.

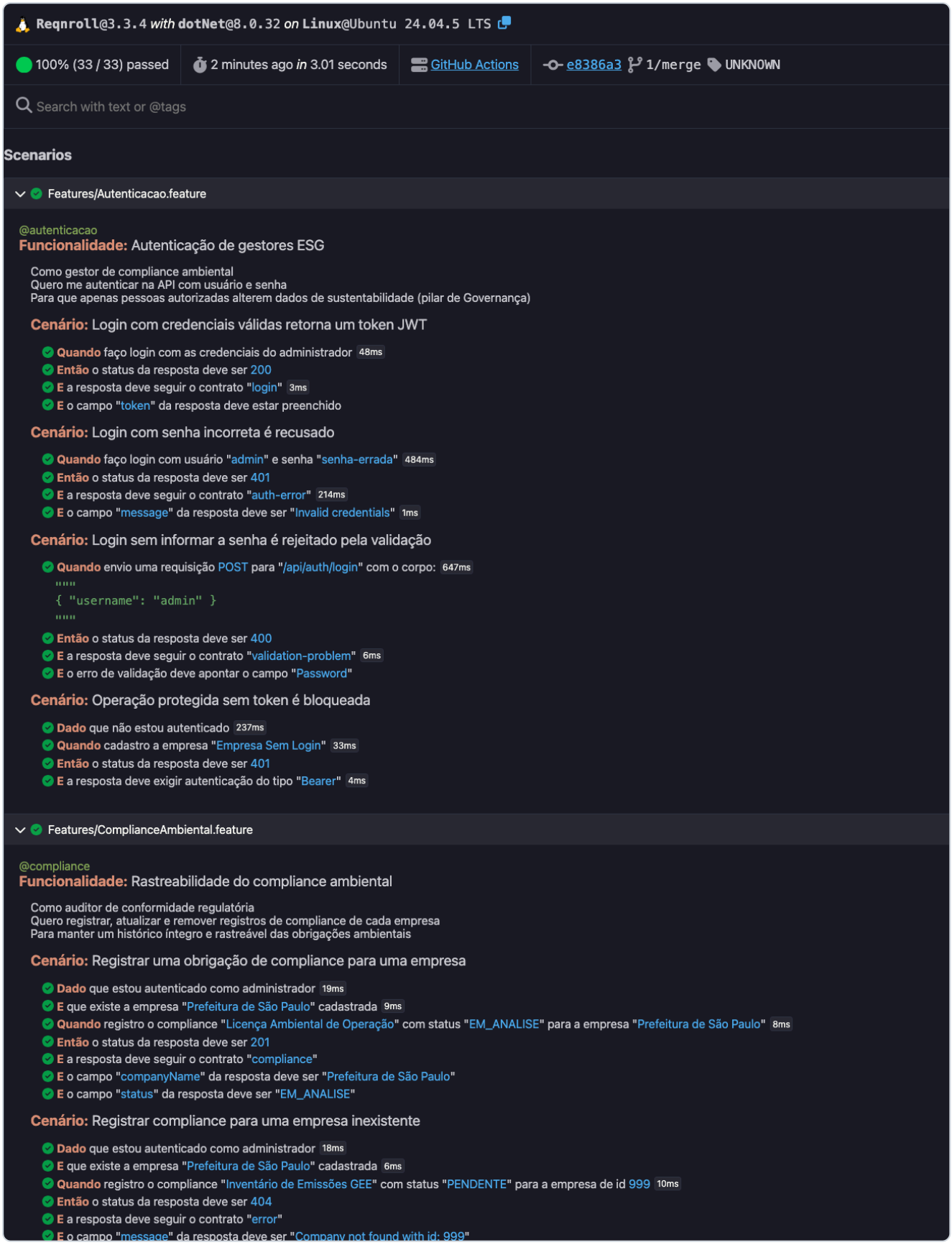


Figura 9: Relatório HTML do Reqnroll **gerado pelo GitHub Actions** (Ubuntu 24.04, .NET 8.0.32): 100% (33/33) aprovados, com cada passo Gherkin e seu tempo.

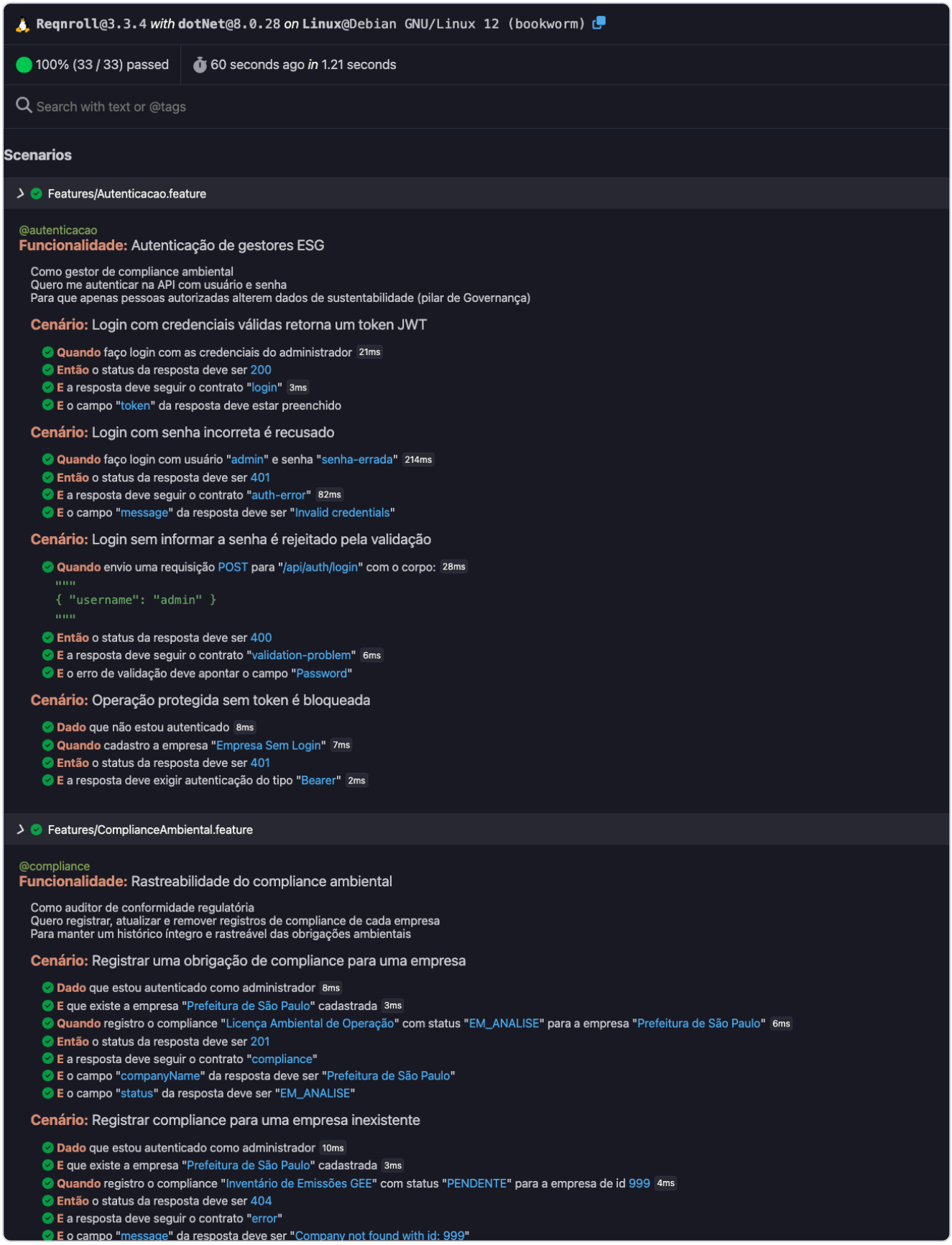


Figura 10: O mesmo relatório gerado na execução local (container Debian, .NET 8).

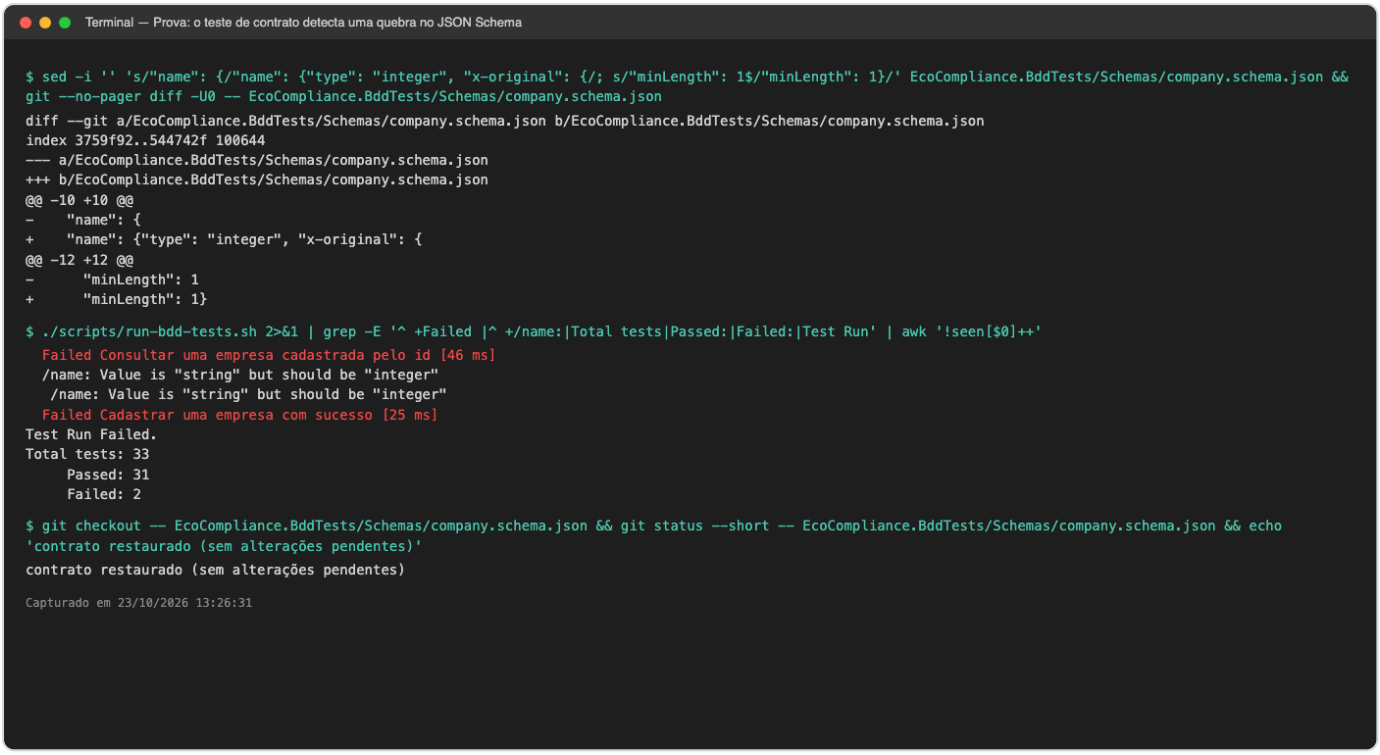


Figura 11: Prova de eficácia dos testes de contrato. O tipo de `name` foi alterado de propósito no schema: os 2 cenários que validam o contrato `company` falharam, apontando o campo exato. O arquivo foi restaurado em seguida.

8. Adaptações na aplicação e desafios

Achado / desafio	Solução
Paginação sem validação: `?pageSize=0` fazia `TotalPages` dividir por zero e `?page=0` gerava *offset* negativo	Regra centralizada (`Pagination.Validate`): `page ≥ 1` e `1 ≤ pageSize ≤ 100` ; fora disso, **400**. Coberta pelo esquema "Paginação inválida é recusada".
Indicador ESG aceitava valor negativo (ex.: –50 tCO2e)	`[Range(0, …)]` em `Value` ; agora a resposta é **400**. É uma regra de integridade dos dados ambientais.
Datas do Oracle saíam sem fuso horário, o que viola o formato `date-time` (RFC 3339) do contrato	`UtcDateTimeConverter` : todas as datas são emitidas em UTC com sufixo `Z` .
Testes dependentes do Oracle da FIAP seriam lentos e instáveis	API real em memória com `WebApplicationFactory` e EF Core InMemory isolado por cenário.
Configurações JWT lidas na inicialização da API	As credenciais e a chave de teste são injetadas com `UseSetting` , aplicado antes da construção do host.
Números em Gherkin pt-BR (`120.5`) interpretados com vírgula decimal	`reqnroll.json` : idioma dos cenários pt-BR e cultura de binding en-US.
Máquina local só com .NET 10: o TestServer do ASP.NET 8 é incompatível com o runtime 10	Execução padronizada em Docker com o SDK 8 (`Dockerfile.tests`), igual ao CI.
Logs de erros esperados (400/404) poluíam a saída	Logging da API desativado durante os testes.

9. Atendimento aos requisitos

Requisito da atividade	Como foi atendido	OK
Mínimo de 3 cenários BDD em Gherkin, positivos e negativos	33 cenários em 5 funcionalidades: 15 positivos e 18 negativos	☑
Testes de API para todas as APIs	13 de 13 endpoints testados via HTTP	☑
Validação de status code	200, 201, 204, 400, 401 e 404	☑
Validação do corpo da resposta (JSON)	Passos *o campo "…" da resposta deve ser/conter*, contagem de itens e erros de validação	☑
Testes de contrato com JSON Schema	11 contratos (draft 2020-12) validados com JsonSchema.Net	☑
Execução simples e documentada, local ou em CI	`./scripts/run-bdd-tests.sh` (Docker) e job no GitHub Actions; documentado no README	☑
.ZIP com testes, Dockerfiles, scripts e README	`EcoCompliance.BddTests/`, `Dockerfile.tests`, `scripts/` e `README.md`	☑
Documentação com cenários Gherkin, prints de configuração e evidências	Este documento: seções 3, 5 e 7	☑